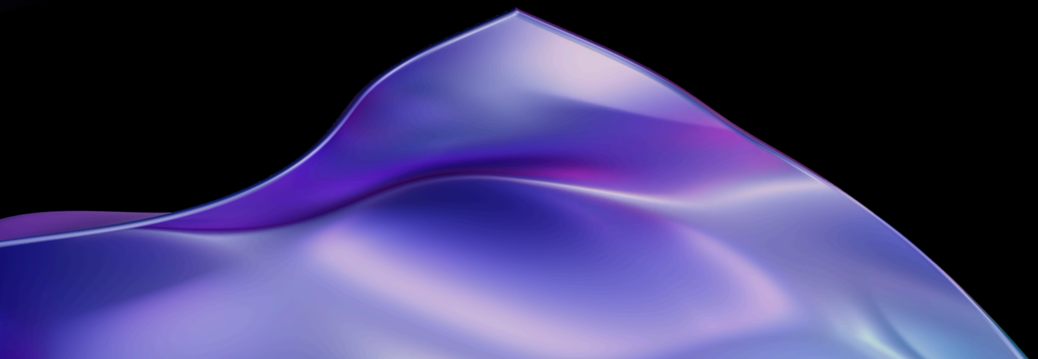
TOOL MANAGEMENT SYSTEM

DOCUMENTATION





Introduction



The Tool Management System (TMS) is designed to simplify inventory management for tools. This system facilitates functionalities like adding, borrowing, returning, and maintaining tools, providing an efficient workflow for administrators and end-users such as faculty and students. Built with SQLite for local storage and a Python or Java-based interface, the TMS emphasizes functionality, offline usability, and simplicity.

- User Roles
- Tool Management
- Transaction Management
- Database
- Graphical User Interface



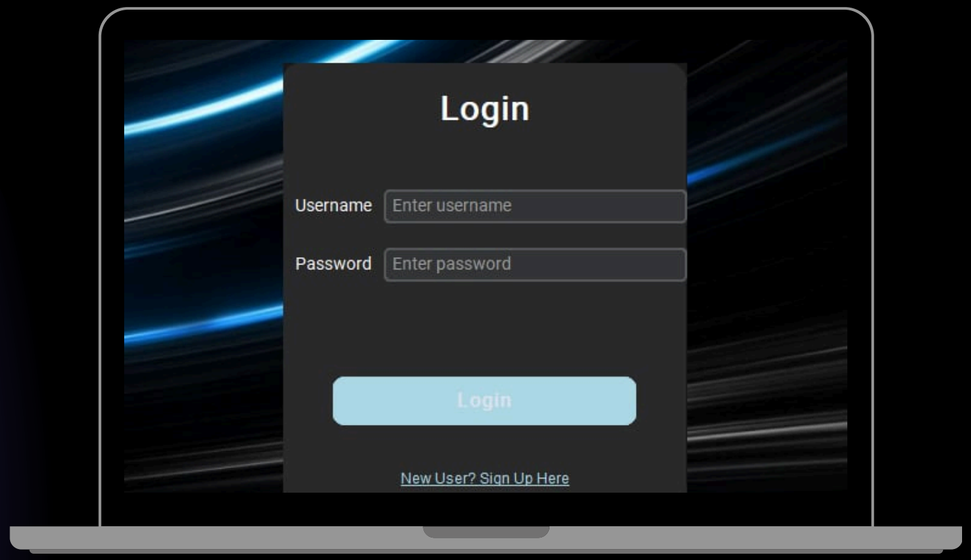
Features

01 User Roles

Admin and Regular Users. Admins Manages the entire inventory system. Performs tasks such as adding, deleting, and updating tools. Interacts with the system to borrow or view available tools.
Limited privileges ensure that inventory management remains under administrative control.

02 Tool Management

- Adding Tools:
  - Admins can add new tools by providing details such as name, category, condition, quantity, and location.
  - Input validation ensures data integrity, reducing errors during entry.
- Editing Tools:
  - Enables admins to modify existing tool information.
  - Supports changes to conditions, quantities, and locations as needed.
- Deleting Tools:
  - Tools that are no longer usable or relevant can be removed.
  - Confirmation prompts help prevent accidental deletions.
- Search Functionality:
  - A search bar allows users to locate specific tools by name or category.
  - Instant search results improve user efficiency.



03 Transaction Management

Borrowing Tools:

- Users can borrow tools by specifying the tool name and quantity.
- Real-time availability checks ensure tools are not over-allocated.
- Borrow transactions are recorded with timestamps for future reference.

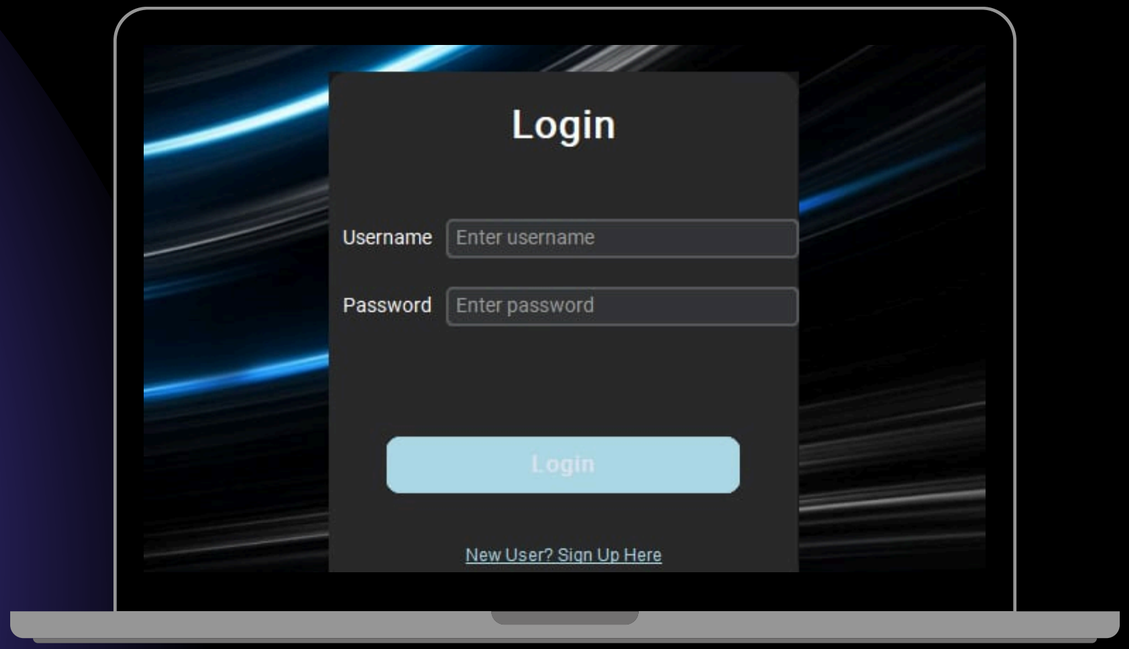
Returning Tools:

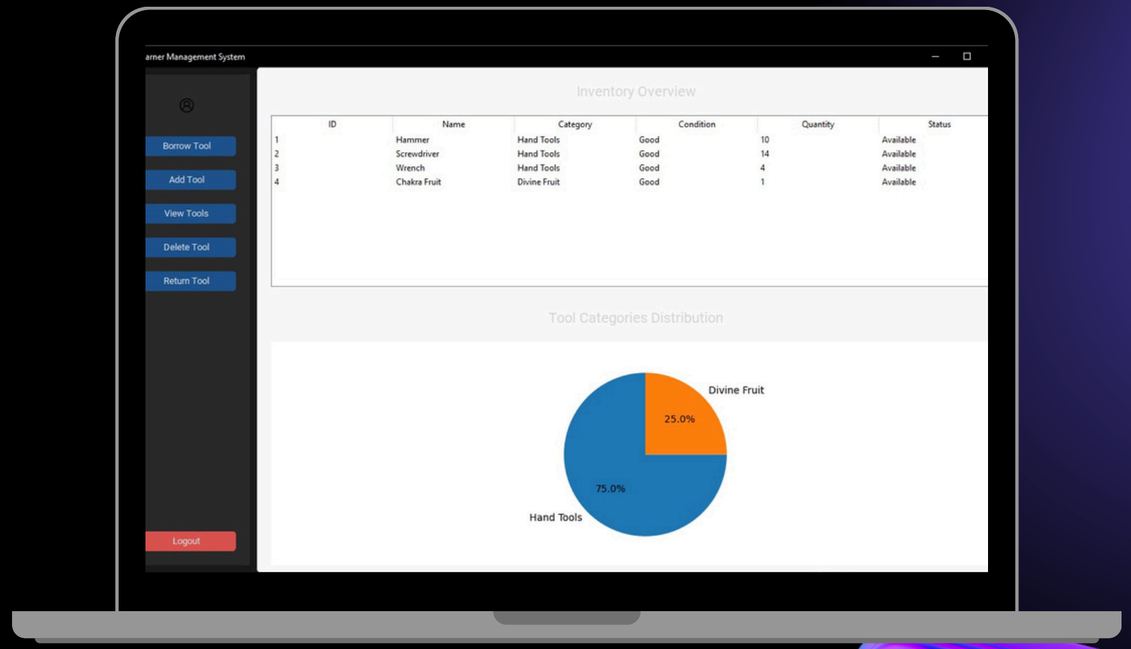
- Simplifies the process of returning tools with real-time updates to the inventory.
- Captures the condition of tools upon return, allowing damaged tools to be flagged.

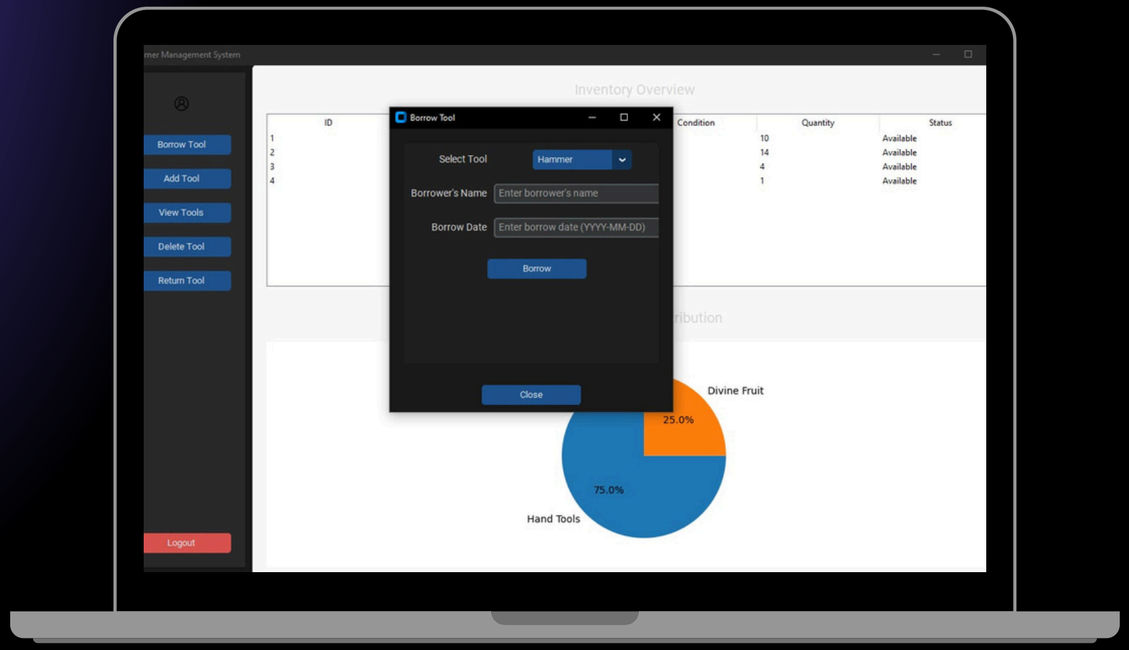
Record Keeping:

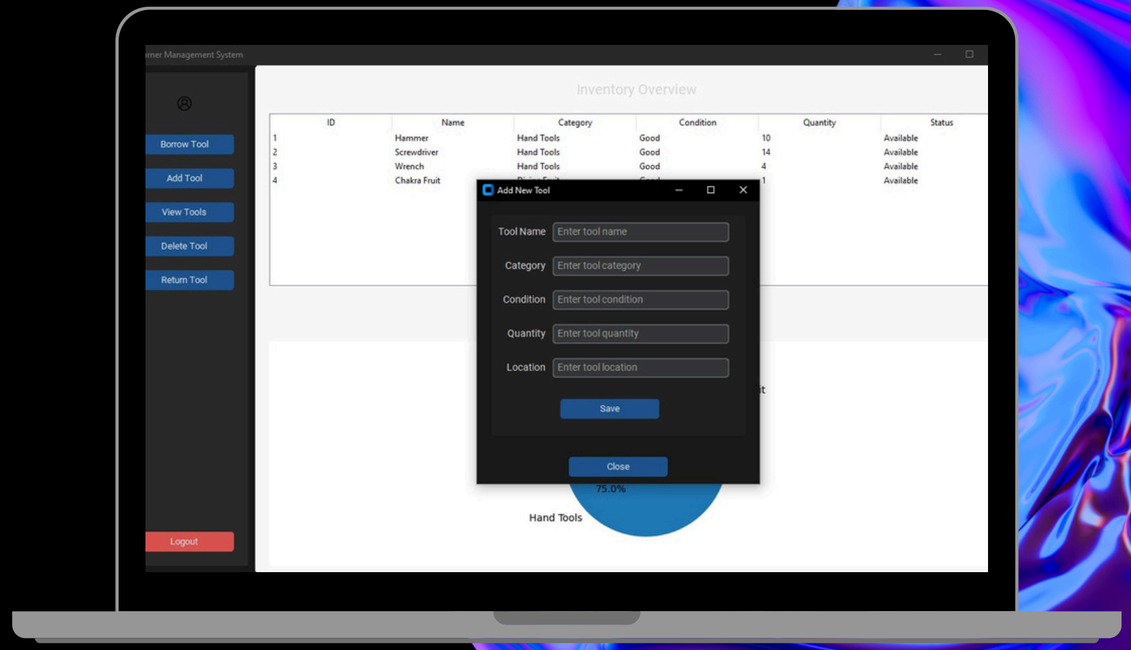
- Comprehensive logs of transactions include details such as user name, tool borrowed, date, and quantity.
- These records are essential for audits and usage analysis.

PROJECT LMS

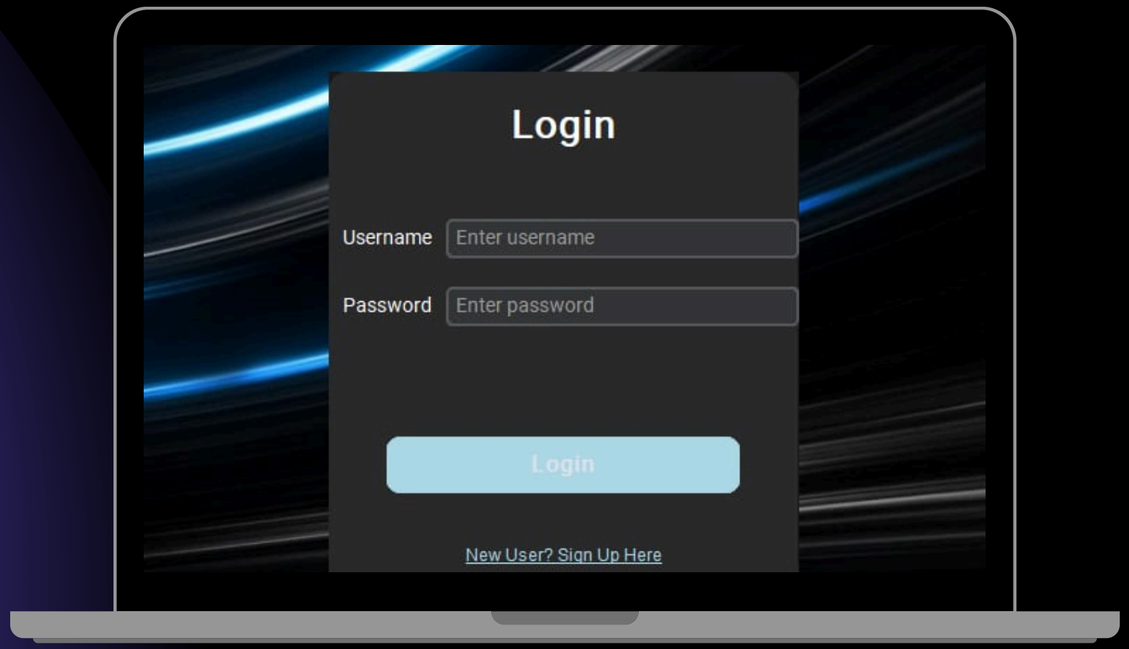

Login
Username
Enter username
Password
Enter password
Login
New User? Sign Up Here

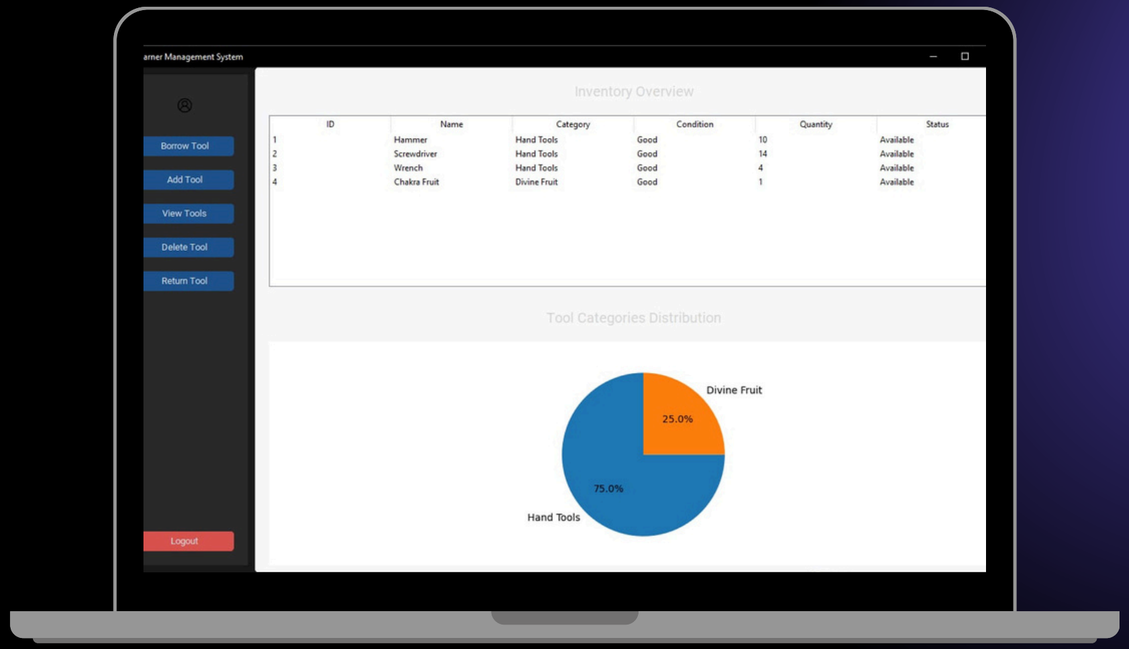

Inventory Overview
Borrow Tool
Add Tool
View Tools
Delete Tool
Return Tool
Logout
ID Name Category Condition Quantity Status
1 Hammer Hand Tools Good 10 Available
2 Screwdriver Hand Tools Good 14 Available
3 Wrench Hand Tools Good 4 Available
4 Chakra Fruit Divine Fruit Good 1 Available
Tool Categories Distribution
Divine Fruit
25.0%
75.0%
Hand Tools

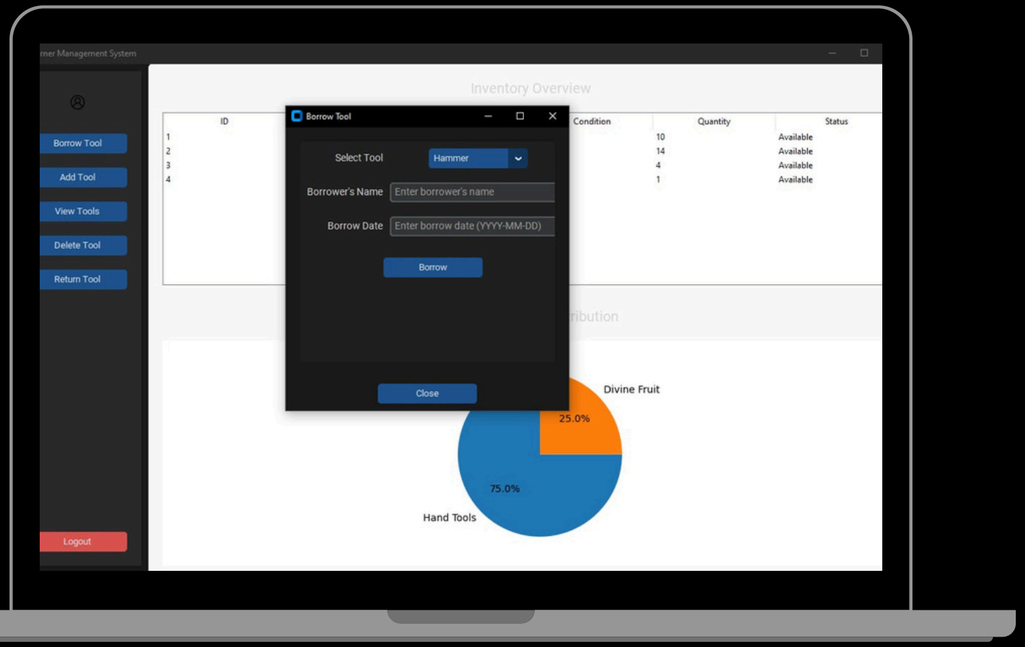

Borrow Tool
Select Tool
Hammer
Borrower's Name
Enter borrower's name
Borrow Date
Enter borrow date (YYYY-MM-DD)
Borrow
Close
Divine Fruit
25.0%
75.0%
Hand Tools


Add New Tool
Tool Name
Enter tool name
Category
Enter tool category
Condition
Enter tool condition
Quantity
Enter tool quantity
Location
Enter tool location
Save
Close
Hand Tools

PROJECT LMS


Login
Username
Enter username
Password
Enter password
Login
New User? Sign Up Here


arner Management System
Inventory Overview
ID Name Category Condition Quantity Status
1 Hammer Hand Tools Good 10 Available
2 Screwdriver Hand Tools Good 14 Available
3 Wrench Hand Tools Good 4 Available
4 Chakra Fruit Divine Fruit Good 1 Available
Borrow Tool
Add Tool
View Tools
Delete Tool
Return Tool
Logout
Tool Categories Distribution
Divine Fruit
25.0%
75.0%
Hand Tools


Inventory Overview
Borrow Tool
Select Tool
Hammer
Borrower's Name
Enter borrower's name
Borrow Date
Enter borrow date (YYYY-MM-DD)
Borrow
Close
Divine Fruit
25.0%
75.0%
Hand Tools
Logout

Manual

The design prioritizes user-friendliness, ensuring even non-technical users can navigate the system easily. Operates entirely offline, making it ideal for environments with limited internet access.

PREREQUISITES

Python 3.x installed on your system.
SQLite pre-installed (comes with Python).

SETUP INSTRUCTIONS

Clone this repository
git clone [https://github.com/cediegreyhat/Project-LMS/.git](https://github.com/cediegreyhat/Project-LMS)
cd Project-LMS

INSTALL REQUIRED PYTHON LIBRARIES:

pip install -r requirements.txt

RUN THE PROJECT:

python3 gui.py



TROUBLE SHOOTING AND CHALLENGES

This section outlines common issues users might encounter while using the Tool Management System (TMS) and provides detailed solutions to resolve them effectively.



Debugging Borrow/Return Conflicts

The system initially faced difficulties handling simultaneous borrow and return transactions, leading to inconsistencies in the database. Implemented transaction locks to ensure that borrow and return actions occur sequentially.

Device Limitations

Not all team members had access to high-performance devices for development and testing. Prioritized lightweight designs for compatibility with less capable hardware. Additionally, tasks were distributed based on device capabilities, ensuring smoother progress.

GUI Elements for Better Usability

Early iterations of the GUI were cluttered and not intuitive for end-users. So we Conducted user testing to identify pain points in navigation and interactions. Redesigned the interface to simplify workflows, grouping related actions (e.g., borrow/return tools) and providing clearer labels and tooltips.



Timestamps for Development Milestones

The project timeline captures significant milestones from initial planning to the final documentation phase. While the entire development process spanned several weeks, specific core tasks were achieved on the following days:

01 Meeting and Planning

- Conducted an initial meeting with stakeholders to define the project's objectives, requirements, and scope.
- Outlined key functionalities, including user roles, tool management, transaction workflows, and database integration.
- Finalized the choice of SQLite for the database

02 Backend Development

- Designed the database schema, including tables for tools, users, and transactions.
- Created functions for adding, editing, deleting, and searching tools in the database.
- Implemented basic role-based access control to differentiate admin and regular users.

03 Basic Interface Design

- Built a skeleton GUI framework to support navigation and basic operations.
- Developed the main dashboard layout, ensuring modularity for future enhancements.

04 Borrow/Return Functionality

- Implemented transaction workflows for borrowing and returning tools.
- Ensured real-time updates to tool quantities in the database.
- Debugged early issues with concurrent transactions and database sync.

05 Search and Reporting

- Integrated a search functionality to allow users to quickly locate tools by name or category.
- Created basic reporting features, including transaction history and inventory summaries.
- Added a pie chart visualization to display tool distribution across categories for admins.

06 Testing and Debugging

- Conducted rigorous testing to identify and resolve bugs in both backend and frontend components.
- Ran test cases for edge scenarios, such as invalid inputs and simultaneous transactions.
- Enhanced error handling mechanisms to ensure user-friendly prompts during failures.

07 Documentation

- Prepared comprehensive user manuals and technical documentation.
- Documented troubleshooting steps for common issues.
- Drafted the project report, including the overview, challenges, and timeline.

Extended Timeframes

- While the core functionalities were developed within the designated days, certain tasks required additional time:
- Debugging borrow/return conflicts took several weeks due to the complexity of transaction locking and real-time updates.
- GUI enhancements were iterative, with refinements made based on user feedback over multiple weeks.

Conclusion and Future Outlook

The Tool Management System (TMS) represents a significant step forward in addressing the logistical challenges faced by organizations that manage physical inventories. Through its streamlined workflows, offline capabilities, and user-friendly interface, TMS empowers users to efficiently manage tools, track transactions, and maintain up-to-date inventories.



Lessons Learned

Collaborative Development

Effective communication and teamwork were critical in overcoming bottlenecks, such as debugging and GUI design.

User-Centered Design

rioritizing user feedback early in the design phase ensured that the system catered to its intended audience's needs.

Importance of Testing

igorous testing at every stage of development proved essential in identifying and resolving potential issues before deployment.

Adaptability

lexibility in responding to unexpected challenges, such as debugging complexities or refining features, ensured the project remained on track.

FULL-STACKED DEVELOPMENT

Lhord Cedrick T. Delos Santos (Team Leader): Full-Stacked Development

DOCUMENTATION AND DEBUGGING

Charles Allen Lacsa: Documentation
Brix Baldiray: Documentation
Dylan Guliquey: Testing and Debug
Joe Ckelvin Barboza: Documentation
Van Hallen Hayag: Documentation

DATABASE INTEGRATION

Luis Fernandez: Basic GUI Development
Warren Dollendo: Basic GUI Development
Dan Angelo Domagsang: Basic GUI Development
Charles Benedict Rafols: Database Development/Debug

GIT NICKNAMES

cediegreyhat - De Los Santos
Gladez-Donuts - Fernandez
Soraya-ping - Dollendo
kneegrow446 - Rafols
Bruux - Baldiray
Chillguy - Lacsa
Van-27 - Van
Lappudum - Dylan
Ckelvin - Joe
redroses - Domagsang